
task-app 30日間カリキュラム

Day 04

ネットに公開

Next.js 15 / TypeScript / Prisma / tRPC

目次

前回の振り返り	5
今日のゴール(Phase 1 修了日)	5
なぜこれを作るのか	7
前提(Day 03 完了していること)	9
今日の見どころ	9
前日からの状態確認	10
実装ステップ一覧	11
Step 1: GitHub 側の完成状態をもう一度確認する(読む目安: 3分)	13
Step 2: 公開前にローカルで npm run build を通す(読む目安: 5分)	15
Step 3: Vercel アカウントを作成する(読む目安: 7分)	18
Step 4: GitHub の task-app リポジトリを Import する(読む目安: 5分)	19
Step 5: Project 設定画面で見るべきポイントを絞る(読む目安: 12分)	21
Step 6: Production Deploy を実行する(読む目安: 5分)	26
Step 7: 発行された URL を開いて最初の公開を体験する(読む目安: 3分)	28
Step 8: シークレットウィンドウと別端末で開く(読む目安: 5分)	31
Step 9: URL を SNS に貼ってみる(読む目安: 3分)	34
Step 10: デプロイ文脈で見る Pro パターン(読む目安: 5分)	35
Step 11: よくあるつまずきを初回公開の順番で切り分ける(読む目安: 5分)	37
Step 12: 今日やったことを、自分の言葉で説明できるようにする(読む目安: 5分)	39
覚えておきたいエッセンス	40
今日のチェックリスト	40
Phase 1(環境構築・即公開)修了 ふりかえり	41
つまずきポイント	43
今日学んだ用語	46

応用課題: ローカルの変更と公開中の画面を見分ける.....	47
理解チェック	47
次回予告	48
次に読むもの	49
奥付	49

Day 04: ネットに公開

Day 01 で土台を立ち上げ、Day 02 でダッシュボードに自分の情報を表示し、Day 03 で GitHub に保存しました。

今日はこのアプリを Vercel(作ったアプリをインターネット上に公開できるホスティングサービスです。読み方: ヴァーセル)を使ってインターネット上に公開します。公開して URL が発行されると自分のパソコン以外からもアプリを開けるようになります。スマホやほかの人のブラウザからもアクセスでき、いま作っているものを実際に見せられる状態になります。

Phase 1(環境構築・即公開)の最終日は新しい機能を追加する日ではありません。すでに作ったアプリを、誰でもアクセスできる状態にすることが今日のゴールです。Vercel での公開手順を、順を追って進めます。

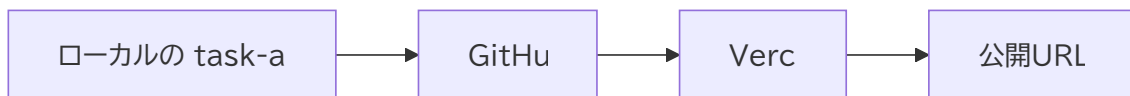


図 1: ローカルから公開 URL までの道筋

この図の矢印は今日 1 日を通る道すじです。左端の手元のコードから右端の公開 URL まで途中に GitHub と Vercel が 1 つずつ挟まります。見てほしいのは Vercel がコードを取りに行く先が自分のパソコンではなく GitHub だという点です。手元でどれだけ直しても GitHub へ送っていない変更は公開されたアプリに出ません。Day 03 で push を覚えたのは今日この矢印をつなぐためでした。

前回の振り返り

Day 03 では `task-app` を自分の GitHub リポジトリへ保存しました。`.env` は送らず、アプリを動かすのに必要なファイルだけを選んで `push` しています。今日はその GitHub リポジトリを Vercel につないで、インターネット上の URL で開ける状態にします。

今日のゴール(Phase 1 修了日)

Day 03 で GitHub へ保存した自分の `task-app` を Vercel につなぎ、実際の URL で公開します。デプロイ(作ったアプリを公開用のサーバーに置いて誰でも使える状態にすること)のボタンを押すだけで終わらせず、Project 設定画面で何を見ればよいかを判断し、公開後に別端末からも開けることを確かめるところまでやります。

Day 01 からここまで進めてきた作業が今日の公開でひとつおりつながります。

Phase 1 の区切りとなる日です。

公開の前にそろえておくもの

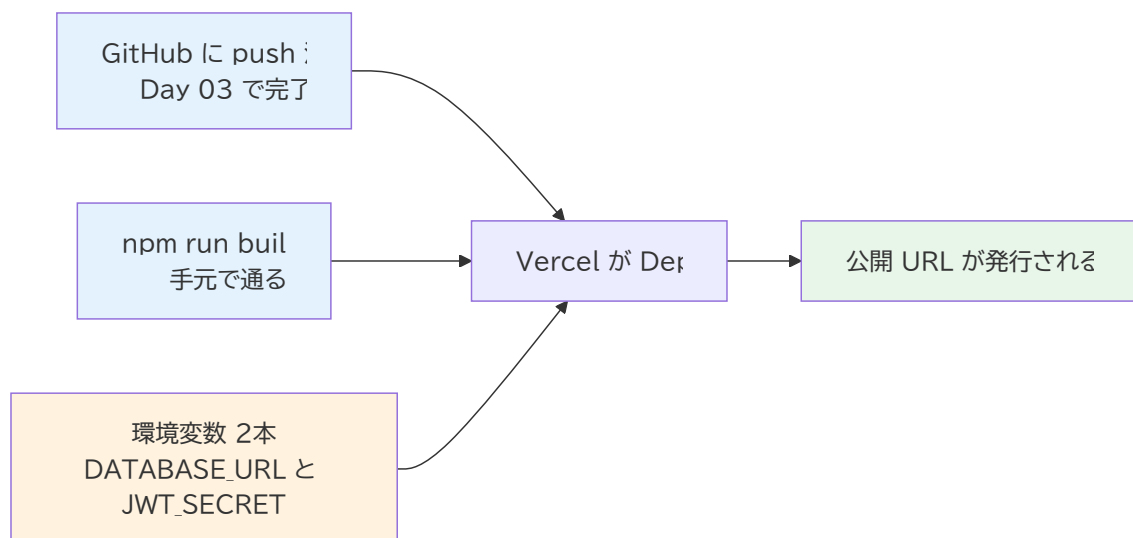


図 2: 公開の前にそろえておくもの

build や環境変数に問題があると Deploy が途中で止まります。GitHub への push を忘れた場合は、古いコードのまま公開されることがあります。Step 1 と Step 2 で最初の2つを、Step 5 で3つ目をそろえます。

発行される URL の形

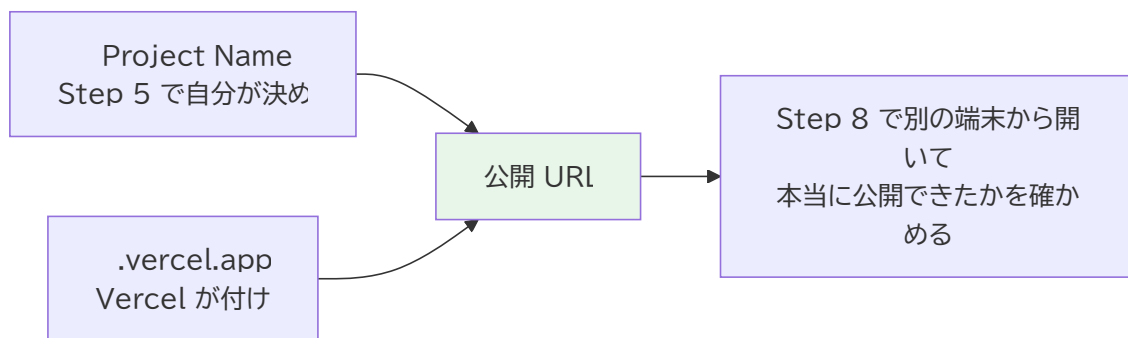


図3: 発行される URL の形

`https://task-app-<自分のユーザー名>.vercel.app` の形になります。数字や記号が混じることもあります。この URL を今日の終わりに人へ送ります。

- ☐ Day 03 の完成状態から作業を再開します。
- ☐ GitHub に `task-app` が push 済みであることを確認します。
- ☐ ローカルで `npm run build` を実行し、公開前の確認をします。
- ☐ Vercel アカウントを作成またはログインします。
- ☐ GitHub リポジトリを Vercel に Import します。
- ☐ 公開先から使う PostgreSQL を Neon(ブラウザだけで PostgreSQL を1つ用意できるサービス)で用意して接続文字列を控えます。
- ☐ Project 設定で `DATABASE_URL` と `JWT_SECRET` を登録し、Production Deploy を実行します。
- ☐ 発行された URL をブラウザで開き、表示を確認します。
- ☐ シークレットウィンドウやスマホでも公開 URL を確認します。

- □ URL を SNS や身近な人に共有して、最初の公開という節目を外へ出します。

なぜこれを作るのか

ここまでのアプリは `http://localhost:3000` でしか開けません。`localhost` は「このパソコン自身」を指す名前なので同じ URL を人に送っても相手の画面には自分のアプリは出てきません。相手のパソコンの中を見に行ってしまうためです。

外から届くようにするには自分のパソコン以外の場所にアプリを置く必要があります。それが今日やるデプロイです。Vercel を使うと GitHub にあるコードを取ってきてビルドして URL を1つ発行するところまでを自動でやってくれます。

そしてもう1つ、今日ここで公開しておく理由があります。**残りの26日ぶん、作った機能をその日のうちに人へ見せられるようになるから**です。30日目にまとめて公開する形だと途中で詰まったときに「そもそも公開できるのか」という不安まで抱えることになります。先に道を通しておけばあとは中身を育てるだけで済みます。

例え話: `localhost` は自分の部屋の机です。デプロイはその机の上のものを店先のショーケースへ移す作業にあたります。中身は同じでも、置く場所が変われば通りすがりの人にも見えます。

新しく学ぶ概念

概念	読み方	役割	例え
Vercel	ヴァーセル	Next.js アプリを手軽に公開できるホスティングサービス	レンタルキッチン。作った料理をお客さんに出せる場所
デプロイ	—	アプリをサーバーに配置して公開する操作	料理をお店のテーブルに出すこと
ビルド	—	ソースコードを本番用の形に変換する処理。 <code>npm run build</code> で走る	仕込みの総仕上げ。お客さんに出せる状態にする
環境変数	かんきょうへんすう	接続先やパスワードを、コードの外から渡す仕組み	レシピに直接書かず、店の金庫に入れておく秘密の材料
接続文字列	せつぞくもじれつ	データベースの場所とログイン情報を1本にまとめた文字列。 <code>DATABASE_URL</code> に入れる	住所と鍵をひとまとめにしたメモ

前提(Day 03 完了していること)

今日は Day 03 の続きから進めます。新しいプロジェクトを作ったり、別の見本コードを取りに行ったりはしません。次の状態になっている前提で進めます。

- 手元に `task-app` ディレクトリがあります。
- `npm install` 済みで `npm run dev` が動きます。
- Day 02 で編集したダッシュボードがローカルに残っています。
- Day 03 で GitHub に push した自分のリポジトリがあります。
- GitHub のブラウザ画面で `task-app` のファイル一覧を見られます。

ここでいちばん大事なのは、今日は自分のリポジトリを、そのまま公開することです。

教材用の別の完成品を使うのではなく、Day 01 から Day 03 まで積み上げた自分の `task-app` を、そのまま Vercel に持っていきます。

本編では新しい機能のコードを追加しません。ただし公開できない個人情報が残っていた場合は、Step 1 で表示名などを直してから進みます。最後の練習問題でも文章を一時的に変えます。中心となる作業は GitHub と Vercel をつなぐ操作と、公開先から使うデータベースを1つ用意することです。Vercel と、データベースを置く Neon の2つで、それぞれアカウントを作る場面があります。どちらも無料の枠から始められます。

今日の見どころ

URL はただの文字列に見えるかもしれませんが。ただ、自分で作ったアプリに URL が付くとできることが広がります。

Day 04: ネットに公開

ローカルで開く `http://localhost:3000` は自分のパソコンの中だけの入り口です。一方 Vercel で公開された URL はほかの端末からも開けます。

今日の終わりには次のことができるようになります。

- 作っているアプリの URL を人に送れます。
- スマホで自分のアプリを開けます。
- SNS に公開したことを投稿できます。
- 明日以降の改善を、公開 URL をもとに考えられます。

30日のカリキュラムのなかでも、作ったものを人へ見せられる最初の区切りです。

前日からの状態確認

まずは Day 03 の最後から今日の流れへのつながりを確認します。Day 03 の締めでは次のように予告していました。

GitHub に保存できたら次はこの履歴を使ってアプリをインターネットに公開します。Day 04 では Vercel につないで、自分の `task-app` を実際の URL で開ける状態にします。今日 GitHub に保存した内容がそのまま公開の土台になります。

今日はこの続きに取り組みます。

GitHub に保存しただけではまだコードが置かれているだけの状態です。そこから Vercel につなぐことで、URL を開けばアプリが見える状態に変わります。この橋渡しを自分でできるようになるのが Day 04 の目的です。

いまの現在地

Day 03 完了時点では次の状態になっているはずです。

- GitHub に `task-app` リポジトリがあります。
- Day 03 で push したブランチに Day 02 までのコードが入っています。
- `README.md` がリポジトリの顔として見えています。
- `.env` は送らず、公開していいファイルだけが保存されています。

ここから今日やるのはVercel に GitHub リポジトリを読み込ませ、公開用のビルドを通し、発行された URL を確認するところまでです。

実装ステップ一覧

ステップ	作業内容	読む時間の目安
Step 1	GitHub 側の完成状態を もう一度確認する	3分
Step 2	公開前にローカルで <code>npm run build</code> を通す	5分
Step 3	Vercel アカウントを作成 する	7分
Step 4	GitHub の <code>task-app</code> リ ポジトリを Import する	5分
Step 5	Project 設定画面で見る べきポイントを絞る	12分
Step 6	Production Deploy を実行する	5分

ステップ	作業内容	読む時間の目安
Step 7	発行された URL を開いて最初の公開を体験する	3分
Step 8	シークレットウィンドウと別端末で開く	5分
Step 9	URL を SNS に貼ってみる	3分
Step 10	デプロイ文脈で見る Pro パターン	5分
Step 11	よくあるつまずきを初回公開の順番で切り分ける	5分
Step 12	今日やったことを、自分の言葉で説明できるようにする	5分

読む時間の合計(仮): 約63分です。

表と各 Step に記した時間は、説明とコードを読む時間の仮の目安です。実測した値ではありません。コードの入力、動作確認、ダウンロードや起動の待ち時間、調べものには別に時間を取ってください。

アカウント登録の確認メールやビルドの待ち時間は、サービス側の状況によって変わります。すでに Vercel のアカウントを持っている場合は、本文の案内に沿ってログインしてください。

Step 1: GitHub 側の完成状態をもう一度確認する(読む目安: 3分)

Vercel に持っていく前に、まずは Day 03 の終点が揃っているかを確認します。

デプロイの失敗はVercel だけが原因とは限りません。GitHub 側に送りきれていない、見たいブランチに変更が入っていないなど、出発点のズレで止まることも多いです。そのためまずは公開前の現在地を確認します。

実行コマンド

```
pwd
git status -sb
git branch --show-current
git remote -v
git log --oneline --decorate -3
```

この5つはどれも状態を読むだけのコマンドです。ファイルやコミットを書き換えないので途中でやめても手元は壊れません。確かめたいのは1点だけです。Vercel はこのあと手元のパソコンではなく GitHub を読みに行くため両者の中身がそろっている必要があります。ここがずれたまま進むと直したはずの表示が公開後も古いまま残ります。しかも原因を Vercel の設定側で探すことになり、初回公開でいちばん迷う形になります。

ここで見たいこと

- `pwd` が `task-app` のルートを指しています。
- `git status -sb` で公開へ含めたい未コミット差分が残っていません。
- `git status -sb` の1行目に `...origin/` があり、送信先のブランチを追跡しています。

Day 04: ネットに公開

- `git status -sb` の1行目に `[ahead 1]` のような未 push 件数が出ていません。
- `git branch --show-current` が Day 03 で push したブランチ名と一致しています。
- `git remote -v` で GitHub の `origin` が見えています。
- `git log --oneline --decorate -3` に Day 03 までの履歴があります。

GitHub のブラウザ画面でも確認する

ターミナルでの確認だけで終わらず、GitHub のリポジトリページもブラウザで開きます。ここで次が見えていれば問題ありません。

- `README.md` が表示されています。
- `src` ディレクトリがあります。
- `src/app/dashboard/page.tsx` が存在します。
- 一番上に表示されているブランチが Day 03 で push したブランチ名と一致しています。

GitHub 側で見えているものが、今日 Vercel が読み込む対象になります。

`git status -sb` の1行目が `## main` のように `...origin/` を含まない場合は送信先を追跡していません。

`git push -u origin "$(git branch --show-current)"` を実行します。

`[ahead 1]` のような表示があれば、その件数ぶんのコミットは手元にしかないため

`git push` を実行します。最後にもう一度 `git status -sb` を実行

し、`...origin/` を含んで `ahead` の無い状態を確認します。作業ツリーが clean でも未 push のコミットは残り得るため、両方を分けて確認します。

Deploy 前に公開してよい内容か確認する

GitHub で `src/app/dashboard/page.tsx` を開き、Day 02 で設定した名前やメッセージを確認します。実名や住所、電話番号など、他人に見られたくない情報があればこのまま Deploy しません。表示名をニックネームなどへ直し、次の3行で変更を GitHub まで送ってから次へ進みます。

```
git add src/app/dashboard/page.tsx
git commit -m "fix: remove private profile details"
git push
```

Vercel の Production URL は Deploy が終わった時点で外から開けるため、この確認は公開前に行います。

この変更は、過去の GitHub の履歴に入った情報までは削除しません。すでに Public へ送った情報を削除する必要がある場合は、GitHub 公式の機密情報削除手順^{*1} に対応と副作用を確認してから進みます。

Step 2: 公開前にローカルで `npm run build` を通す(読む目安: 5分)

ここは今日の大事な一手です。

Vercel が build してくれるので先にプッシュしてから確認すればよい、と思いがちですがこの進め方はあとで手間が増えやすいです。

1. <https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/removing-sensitive-data-from-a-repository>

Day 04: ネットに公開

Vercel の失敗ログも確認できますがまずローカルで `npm run build` が通るかを確認しておくで公開前にエラーを切り分けやすくなります。公開前にローカルで一度ビルドを通す習慣をつけておきます。

実行コマンド

```
npm run build
```

`npm run dev` はコードを書き換えたときの表示や実行結果を確かめるコマンドです。TypeScript の型検査は行いません。画面が開いても、型の誤りに気付かない場合があります。

`npm run build` は本番用のファイルを作り、その途中でプロジェクト全体の型を確認します。Vercel が公開時に実行するのも、この `build` です。型だけを確認したいときは `npm run type-check` を使います。

このスターターでは、型検査とコードの書き方の検査を分けています。使っていない変数などは `npm run lint` で Biome に確認させます。`build` の成功だけで両方を確認したことにはなりません。

期待する結果

次は出力例です。環境によって表示は変わります。

なお `npm run build` はその中で `prisma generate` も実行します。そのため ▲ `Next.js` の行より先に Prisma の行が数行流れます。

`Generated Prisma Client` という行が出ていればそこは正常です。ページ数の `(6/6)` の部分はその時点のページ数によって変わります。ここまでの3日ぶんを素直に進めた状態ならこの数字になります。

```
▲ Next.js 15.5.24
- Environments: .env

Creating an optimized production build ...
✓ Compiled successfully in 13.2s
Linting and checking validity of types ...
Collecting page data ...
✓ Generating static pages (6/6)
Finalizing page optimization ...
Collecting build traces ...
```

最後まで止まらず完走すれば今日の公開にかなり自信を持って進めます。

この行が順に流れるのは build が決まった順番で進むからです。ページ数や秒数は手元の状態で変わります。ソースの変換、型の確認、ページごとの下ごしらえ、最後に最適化、という並びになっています。だから途中で止まったときは最後に出た行の名前がそのまま調べる場所を教えてください。

`Linting and checking validity of types` の行で止まればまず型の書き間違いを疑います。`Generating static pages` まで進んでいたなら型ではなくページ側の処理でエラーが起きています。

もしここで止まったら

落ち着いてまずローカルの build エラーを直しましょう。

赤い文字がたくさん出ても全部を読む必要はありません。見るのは次の3つです。

- 最初に出た赤い行。あとに続くエラーはその巻き添えで出ていることが多いです。
- `Error:` や `Type error:` のあとに書かれている内容

- `./src/...` のようなファイル名と、その横に付いている行番号

なぜ最初の1行を優先するかというと build は途中で1か所つまずくとその先の処理も連鎖で止まるからです。だから下の方のエラーを追いかけるより一番上のエラーを直す方が早いです。ファイル名と行番号が分かれば直す場所は「どのファイルの何行目か」まで絞れます。まずはそのファイルのその行を開いて直前に自分が変えたところと見比べます。多くは書き間違いや消し忘れが原因です。

この時点なら Vercel の設定が悪いのか、コード自体が build できていないのか、原因の切り分けがとてもやりやすいです。

逆にここを飛ばすと Vercel 側の UI とログを見ながら原因を探すことになって初回公開ではかなり迷いやすいです。

今日は最速でボタンを押す練習ではなくて公開までの流れを自分で再現できるようになるのが狙いです。

Step 3: Vercel アカウントを作成する(読む目安: 7分)

GitHub 側の土台とローカル build が確認できたら次は公開先の Vercel に入ります。

Vercel は Next.js とかなり相性がいい公開基盤です。今回の `task-app` みたいな App Router 構成でも、初回の導線がかなり分かりやすいです。

やること

1. ブラウザで `https://vercel.com` を開きます。
2. `Sign Up` か `Continue with GitHub` を選びます。
3. GitHub アカウントと連携してログインします。

ここで GitHub 連携を選ぶ理由

今日は Day 03 の続きで、GitHub にある自分のリポジトリをそのまま公開したいからです。

GitHub 連携にしておくと Vercel 側でリポジトリ選択がかなり素直になります。あとで更新を push したときも、再デプロイの流れがつながりやすいです。

初回ログインで見かけやすい画面

初回だと次のどれかが出ることが多いです。

- Import Git Repository
- Add New...
- Continue with GitHub
- GitHub 連携の権限確認画面

文言は少し違っていてもよいです。大事なのは自分の GitHub リポジトリ一覧が見える入口まで進むことです。

期待する結果

- Vercel にログインできています。
- リポジトリ一覧に `task-app` が見えています。

Step 4: GitHub の `task-app` リポジトリを Import する(読む目安: 5分)

ログインできたら Vercel に Day 03 で作った GitHub リポジトリを読み込ませます。

ここで選ぶのは自分の `task-app` です。

進め方

1. Vercel のダッシュボードで **Add New...** を押します。
2. **Project** を選びます。
3. GitHub リポジトリ一覧から **task-app** を探します。
4. 見つかったら **Import** を押します。

リポジトリが見つからないとき

だいたい次のどちらかです。

- GitHub 連携の権限がまだ足りていません。
- Vercel にリポジトリアクセスを許可していません。

この場合はリポジトリ一覧の近くにある **Adjust GitHub App Permissions** (環境によっては **Configure GitHub App**) を押します。GitHub の設定画面が開きます。**Repository access** の項目で **All repositories** を選ぶか、**Only select repositories** から **task-app** を選んで保存します。Vercel の画面に戻ると一覧に **task-app** が出てきます。

ここでコードを直す必要はありません。Vercel から見えないのは多くの場合「どのリポジトリを見てよいか」という許可がまだ渡っていないだけだからです。直す場所も、コードではなく GitHub 連携の許可範囲になります。

初回でここに数分かかるのは普通です。焦らなくてよいです。

ここで意識してほしいこと

Import するのは今日のために新しく用意した何かではなくて Day 01 から積み上げてきた自分のリポジトリです。

この感覚があると「公開は別工程」ではなくて「作ってきたものの自然な続き」として理解しやすくなります。

期待する結果

- `task-app` の行で `Import` を押せて Project 設定画面に進めています。

Step 5: Project 設定画面で見るべきポイントを絞る(読む目安: 12分)

`Import` を押すと Project 設定画面に進みます。

ここで項目がいろいろ出てくると初回はちょっと身構えるかもしれませんが。でも Day 04 で全部を理解し切る必要はありません。

今日まず見るべきポイントはかなり絞れます。

まず見る場所

- Project Name
- Framework Preset
- Root Directory
- Build and Output Settings
- Environment Variables

今日の `task-app` での基本判断

- Project Name: `task-app` か、被りを避けたいなら少し変えても OK
- Framework Preset: `Next.js`
- Root Directory: リポジトリのルート
- Build Command: デフォルトのままで問題ないことが多いです。
- Output Directory: デフォルトのままで問題ないことが多いです。

Day 04: ネットに公開

- Environment Variables: 今日は `DATABASE_URL` と `JWT_SECRET` の2つだけ設定します。

Environment Variables には2つだけ設定する

Day 04 のスコープはまず自分のアプリを公開 URL で見られるようにするところです。

ただし `DATABASE_URL` と `JWT_SECRET` の2つは今日必要です。配布コードは起動時に環境変数を検証する作りになっていて `DATABASE_URL` (有効な URL 形式) と `JWT_SECRET` (32 文字以上) が無いとエラーを出して止まります。Vercel 上でも同じ検証が走るため空のままではデプロイしたアプリが起動しません。

新語の表で「環境変数 = 店の裏の金庫」と例えました。今日はその金庫に、鍵を2本だけ入れます。それ以外の値は本番公開を仕上げる Day 30 で扱います。

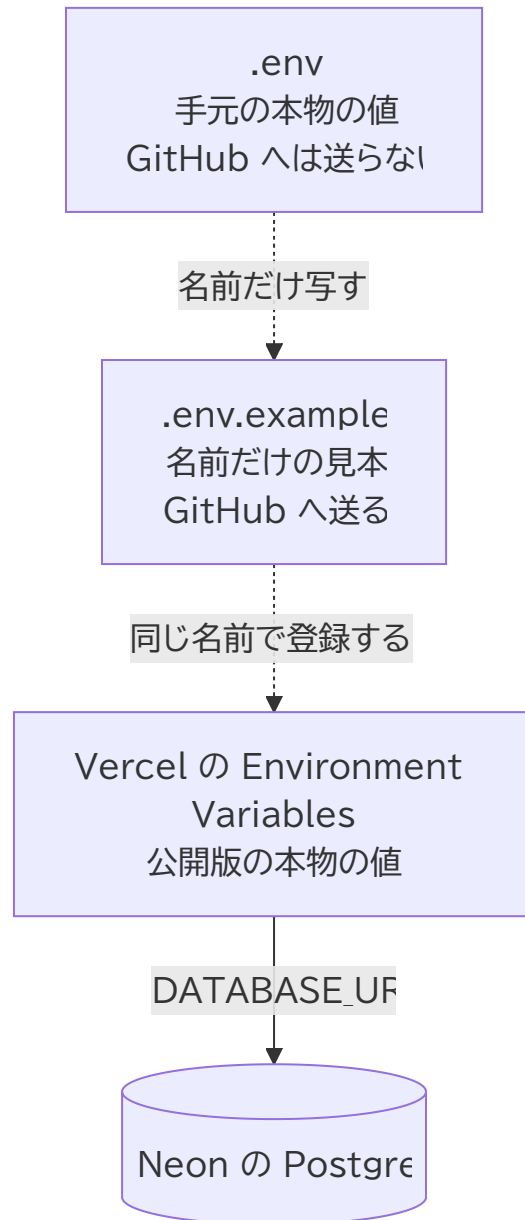


図4: Environment Variables には2つだけ設定する

同じ名前の設定値が3か所に分かれて置かれます。本物の値が入るのは `.env` と Vercel の2か所だけで、GitHub に載るのは名前だけの見本です。実線は値が実際に使われる向き、点線は名前をそろえる作業を表します。

JWT_SECRET を用意する

手元のターミナルで、次のコマンドを実行します。

ターミナル(どこでもOK)

```
openssl rand -base64 48
```

長いランダムな文字列が1行表示されます。これが JWT(ログイン状態の証明に使う署名付きデータ)の署名用シークレットになります。表示された文字列をコピーして Environment Variables の欄に Key を `JWT_SECRET`、Value をその文字列として追加します。Environment は `Production` を必ず選び、追加内容を保存します。

DATABASE_URL を用意する

Vercel から接続できる PostgreSQL が1つ必要です。無料枠のあるサービスに Neon^{*2} や Supabase^{*3}(PostgreSQL を無料枠から使えるクラウドサービス)があります。どちらかでアカウントを作ってデータベースを1つ作成すると

`postgresql://` で始まる接続文字列(ホスト名やパスワードをまとめた1行の接続情報)が発行されます。それを Key `DATABASE_URL` の Value として追加します。

ここでは Neon を例に、実際の手順を書きます。

1. `https://neon.tech` を開き、`Sign up` から GitHub アカウントで登録します。
2. 登録後の画面で `Create project` を押します。
3. プロジェクト名はそのままよいので `Create` を押します。

2. <https://neon.tech>

3. <https://supabase.com>

4. 作成直後に出る `Connection string` の欄で、`postgresql://` で始まる 1 行をコピーします。(欄が見当たらないときは左メニューの `Dashboard` から `Connect` を押す)
5. コピーした 1 行を、Vercel の Environment Variables に Key `DATABASE_URL` として追加します。

`DATABASE_URL` も Environment に `Production` を選んで保存します。

Vercel の環境変数は Production・Preview・Development で適用先が分かれているため、値が登録済みでも Production が対象外なら次の Deploy では使われません。

画面の見た目はサービスの更新で変わります。ボタンの名前が違っていてもやることは「プロジェクトを作る」「接続文字列をコピーする」の 2 つだけです。Supabase を使う場合も、この 2 つに当たる操作を探してください。

本番データベースの本格的な運用(テーブル作成やデータ投入)は後日の Day で扱います。今日は「起動時の検証を通すための接続先」が 1 つあれば十分です。

初回に迷いがちなところ

Project Name

公開 URL にはこの名前がよく入ります。たとえば `task-app-kouiso` ならそれに近い URL が発行されやすいです。

SNS に貼ることを考えると短くて分かりやすい名前はけっこう効きます。

Framework Preset

Next.js のプロジェクトなのでここは `Next.js` が選ばれていれば大丈夫です。

Vercel はこの選択を見てどの build コマンドを走らせ、どのフォルダを公開用に配るかを決めます。`Other` のまま進むと Next.js 用の手順が使われず、build は成功したのに開いた URL が真っ白、という結果になりがちです。設定を1つ取り違えただけなのに画面には何も出ないため原因にたどり着くまで時間がかかります。Import した直後に一度だけ見ておく価値があります。

Root Directory

今日の教材ではリポジトリ直下にアプリがある想定です。サブディレクトリ構成ではないのでルートのまま進めます。

期待する結果

- Framework Preset が `Next.js` になっています。
- Environment Variables に `DATABASE_URL` と `JWT_SECRET` の2つが入っています。
- 2つとも Environment の対象に `Production` が含まれています。

Step 6: Production Deploy を実行する(読む目安: 5分)

設定が確認できたらいよいよ公開です。

`Deploy` を押します。この瞬間に Vercel が GitHub からコードを取得して build して公開 URL を用意し始めます。

ここで起きていること

画面の向こうではざっくり次の流れが走っています。

1. GitHub から `task-app` のコードを読みます。
2. 依存関係をインストールします。
3. `npm run build` 相当の build を走らせます。

4. 生成物を公開用に配置します。
5. URL を発行します。

つまり Day 03 で GitHub に保存した履歴が今日 Vercel に読まれて Day 04 の公開につながっています。

このつながりが見えたら GitHub と Vercel の役割も整理しやすいです。

待っている間に見るべきところ

デプロイ中は進捗ログがよく流れます。

ここで見ておきたいのは次の3つです。

- Install が進んでいるか
- Build が通っているか
- 最後に **Ready** っぽい表示が出るか

成功のイメージ

文言は多少違ってほしいこういう雰囲気です。

```
Building
Deploying
Ready
```

この **Ready** が見えたら今日の最初の公開は成功にかなり近いです。

3つの語はVercel が内部で進めている工程の見出しです。**Building** の間はコードを本番用に変換しています。**Deploying** へ変わると出来上がったものを世界中の配信サーバーへ配り始めます。**Ready** はその配布が終わった合図です。ここまで

Day 04: ネットに公開

来て初めて URL を開いたときに中身が返ってきます。赤い文字で `Building` のまま止まったときはまだ1台にも配られていません。作りかけのアプリが外から見える状態になることはないので落ち着いてログを読んで大丈夫です。

Step 7: 発行された URL を開いて最初の公開を体験する(読む目安: 3分)

デプロイ完了後、Vercel が Production URL を表示してくれます。

たとえば次のような形です。

```
https://task-app-kouiso.vercel.app
```

実際の URL は人によって違います。でもこの形式に近いものが出たら、それが今日の成果物です。

`.vercel.app` の手前にある部分は Step 5 で決めた Project Name から作られます。名前に本名や勤務先の呼び名を入れるとそれが URL の一部として誰の目にも触れます。そしてこの URL はアカウントを持たない人でも開けます。いまの `task-app` にログイン画面はまだなく、認証を足すのは Day 05 からです。つまり今日の時点では画面に出ているものは全部見られる前提で扱います。

まずはブラウザで開く

表示された URL をクリックして自分のアプリを開きましょう。

ここで見えた瞬間、かなりテンション上がるはずです。

さっきまで `localhost` という自分のパソコンの中だけの画面だったものがいまは公開された URL に置かれています。外の誰からも開ける場所に移りました。見た目は同じでも、届く範囲がまるで変わっています。

最初に出るのは Day 01 で作ったトップページです。Day 02 のメッセージは `/dashboard` にあるのでトップページの「ダッシュボードへ入る」を押して移動してください。

確認ポイント

- ページがエラーにならず表示されます。
- Day 01 から積み上げた見た目が見えます。
- 「ダッシュボードへ入る」を押すと Day 02 で作った自分用メッセージが見えます。
- GitHub 上の最新状態とズレていません。

Day 04: ネットに公開



図5: Day 02 で作ったダッシュボード。ヘッダーに My Dashboard、本文に自分あてのメッセージ、右側に3つのカードが並んでいる

次のデプロイからは手元で `npm run build` を実行してから `npm start` で立ち上げるとビルド済みのアプリを本番と同じモードで動かせます。`npm run dev` が開発中の確認用なのに対して `npm start` は「ビルドが通るか、ビルド後も動くか」を先に確かめる予行演習です。ここで動けばVercel 上のビルドエラーの多くを事前に潰せます。`npm start` は `npm run dev` と同じ 3000 番を使うので先に `npm run dev` を Ctrl+C で止めてから実行します。

ただしこれで確認できるのはビルドが通ることまでです。Vercel 側の設定・環境変数・外から届くかどうかはローカルでは何も証明されません。今日のゴールは「ネットに公開」なので完了の判定は必ず実際の公開 URL で行います。とくに、自分のパソコンと同じ Wi-Fi ではない回線(スマホのモバイル回線など)から URL を開けたら本当に公開できた証拠になります。次の Step 8 でそれを確認めます。

上のキャプチャは Day 02 で載せたものと同じ画像です。`npm run dev` でも `npm start` でも、公開した URL でも、画面の見た目は変わりません。変わるのは届く範囲だけなのでどこで開いても同じ絵になります。別々に撮って並べると「何かが違うはず」と探させてしまうので同じ画像を指しています。

最初に見てほしい感覚

この瞬間は細かいデザイン反省より先にまずひとつ受け取ってください。

自分のアプリに、外から入れる URL が付いた

これが Day 04 のど真ん中です。

Step 8: シークレットウィンドウと別端末で開く(読む目安: 5分)

公開 URL をブラウザで1回見て終わり、にしないのが大事です。

本当に公開されているなら別セッションや別端末でも見えるはずです。

まずやること

1. シークレットウィンドウを開きます。
2. さっきの Vercel URL を貼ります。
3. 同じ画面が見えるか確認します。

次にやること

1. スマホを手元に出します。
2. スマホの Wi-Fi をいったん切ってモバイル回線にします(自宅ネットワークの外から開くため)。
3. URL を自分に送ります。
4. スマホのブラウザで開きます。

ここで見ておきたいこと

- ちゃんと読み込めるか
- 画面が極端に崩れていないか
- 自分以外の環境でも URL が生きているか

PC のいつものブラウザで見えるだけだとログイン状態やキャッシュに助けられている可能性もあります。だからシークレットウィンドウとスマホ確認は初回公開でかなり価値が高いです。

この2つは証明している中身が違います。シークレットウィンドウで分かるのは保存済みのログイン情報やキャッシュ(一度読んだデータを手元に貯めておく仕組み)に頼らず表示できることです。モバイル回線のスマホで分かるのは自宅のネットワークの外からでも届くことです。前者だけでは同じ回線の中でしか見えない状態と区別が付きません。両方そろってようやく他人の環境でも開けると言い切れます。

スマホで開けたときの意味

ここでようやく、「自分の生活圏の中で実際に使えるもの」に一段近づきます。

朝スマホで見てもよいし、誰かにその場で見せてもよいです。この実感はローカルだけではなかなか出にくいです。

公開 URL を共有する前に知っておくこと

Step 8 で確認したとおり、この URL は自分以外の端末からも開けます。言いかえると URL を知っている人なら誰でもアプリを開けるということです。今の

`task-app` にはログイン機能がまだありません。認証は Day 05 から作ります。そのためいまの公開 URL は認証なしの完全公開状態です。

広く共有する前に、次の3点を確認しておきます。

画面に個人情報を出さない

Day 02 でダッシュボードに表示した名前やメッセージは公開 URL でもそのまま見えます。Step 1 で公開してよい内容へ整えたことを、共有前にもう一度確認します。

本物のシークレットは GitHub に置かない

Day 03 では `.gitignore` と `.env.example` の役割を確認しました。本物の値はリポジトリに送らず、見本だけを共有するという線引きです。API(プログラム同士が機能やデータをやり取りするための窓口)のキーやデータベース接続情報のようなシークレットを、GitHub には含めません。こうした値は公開せず、Vercel の Environment Variables に直接登録します。今日設定した `DATABASE_URL` と `JWT_SECRET` も、この考え方で GitHub ではなく Vercel 側に置いています。残りの本番設定は本番公開を仕上げる Day 30 で扱います。なお `NEXT_PUBLIC_` で始まる環境変数はブラウザにも渡ります。秘密の値には使いません。

公開後の更新

連携後は GitHub への push で新しいデプロイが始まります。公開用に設定したブランチへ push すると、ビルド成功後に公開中の画面が更新されます。別のブランチでは確認用の Preview URL が作られます。

非公開に戻す方法があることを知っておく

公開を止める操作もあります。ただし、Deployment を1件削除しても、ほかの Deployment まで消えるわけではありません。Project 自体を削除すると、その Project の全 Deployment に加えて環境変数などの設定も削除されます。学習を続けるため、ここでは削除しません。

Day 04: ネットに公開

必要になったら Vercel の Project 削除手順^{*4} で対象と失われる設定を確認します。一度見られた内容や保存されたコピーまでは取り消せないなので、公開前の確認が大切です。

公開 URL は `*.vercel.app` という形で、検索エンジンにも見つかる可能性があります。だからこそ、共有する前に、この画面が誰に見られても大丈夫かを一度だけ確かめておきます。

Step 9: URL を SNS に貼ってみる(読む目安: 3分)

ここが今日のいちばんの見どころです。

せっかく公開 URL が出たのに自分の中だけで閉じてしまうのはもったいないです。

公開範囲は自分で決めてよいです。大きく出たくなければ身近な友だちや家族に送るだけでも十分です。

でももし出せそうなら今日はぜひ URL を SNS に貼ってみてください。

共有文のたたき台

Day 04 で自分の task-app を初公開できました。
まだ土台だけですが、URL を持てたのがうれしいです。
次は Day 05 でログイン画面を作ります。
`https://your-task-app.vercel.app`

4. <https://vercel.com/docs/projects/managing-projects#deleting-a-project>

ここに貼る URL は必ず Step 8 で別回線から開けたものを使います。手元のブラウザでしか試していないアドレスを送ると相手の画面では開けない、ということが起こります。文面のほうはうまく書こうとしなくて大丈夫です。読む人が知りたいのは完成度ではなく、いま何を作っているのかという1点だからです。

書くときのコツ

- 完成品みたいに見せすぎなくてよいです。
- 「今日ここまで来た」が伝われば十分
- Day 01 から積み上げてきた流れを一言添えると良いです。

なぜここを勧めるのか

公開って出した瞬間に初めて現実感が出るものです。

SNS や人に見せることで、「次に何を良くしたいか」も見えてきます。Day 05 以降のモチベーションにもかなり効きます。

Phase 1 の節目としてここは素直に味わってよいです。

ここで確認したいこと

公開 URL を、自分以外の誰かが開ける場所に1回出せていれば十分です。SNS でも、知人への共有でもかまいません。

Step 10: デプロイ文脈で見る Pro パターン(読む目安: 5分)

Pro パターンで書こう(build を通してから公開する)

ここまでで、Vercel に公開する流れはつかめました。ただし公開前の順番の組み方にはまだ差の出るところが残っています。なぜその順番にするのかを、Before/After で見比べます。

Before(動くかどうかを Vercel 任せにする)

```
# ローカルでの確認を飛ばして、そのまま Vercel に Import する  
# うまくいかはデプロイ結果を見てから考える
```

この進め方の問題点:

- build 失敗の原因がコードの問題なのか Vercel 設定の問題なのか切り分けにくいです。
- デプロイの待ち時間ごとに確認コストが発生して初回公開ほど迷いやすいです。
- 「公開前に自分で品質を確かめる」流れが身につきにくいです。

After(ローカル build を通してから公開する)

```
npm run build  
# 通ったことを確認してから Vercel に Import / Deploy する
```

この進め方の強み:

- コード側の build 問題を先に潰せるので Vercel 側では設定確認に集中しやすいです。
- 失敗時の原因切り分けが早くなって公開フロー全体の再現性が上がります。
- 「公開前に一呼吸置く」習慣がつくので今後の本番デプロイでも崩れにくいです。

覚えておきたいエッセンス

デプロイは「押してから祈る」よりローカル build を通してから公開するほうが強いです。

ネットに公開するほど事前確認の一手があとで効いてきます。

ここで確認したいこと

「公開の前にローカルで `npm run build` を通す」という流れを、自分の言葉で説明できれば十分です。

Step 11: よくあるつまずきを初回公開の順番で切り分ける(読む目安: 5分)

Vercel 初回公開は詰まると全部が難しく見えやすいです。でも実際はだいたい次のどこかで止まっていることが多いです。

- GitHub 側の状態
- ローカル build
- Vercel 連携権限
- Project 設定
- 公開後の確認

ここでは今日の流れに沿って見直し順を置いておきます。

`task-app` リポジトリが Vercel に出てこない

まず見るのは GitHub 連携権限です。

- Vercel に GitHub 連携が通っているか
- 対象リポジトリへのアクセスが許可されているか
- ブラウザで GitHub 側のリポジトリがちゃんと見えているか

リポジトリ自体が GitHub に見えていなければ Vercel 側でも見えません。だから出発点から順に戻るのが早いです。

Deploy が build エラーで止まる

まずローカルでこれをやります。

```
npm run build
```

ここで同じエラーが出て止まるなら原因はコード側の可能性が高いです。

逆にローカル build は通るのに Vercel で止まるなら設定や環境差分を疑いやすいです。

公開 URL は出たのに画面が思ったのと違う

次を確認しましょう。

- Day 03 で push したブランチに最新コードが入っているか
- Vercel が読んでいるブランチが意図どおりか
- デプロイ日時が最新か

初回は「見ているコードのバージョンがズレている」だけのことも多いです。

PC では見えるのにスマホで崩れる

まずは焦らず、どこまで崩れているかを観察しましょう。

- 文字が切れているか
- 横スクロールが出ているか
- そもそも読み込みエラーなのか

詳しいレイアウト修正は今日の作業には含めません。公開した画面を別の端末でも開き、文字の欠けや横スクロールがないか確認しておきましょう。

共有するのがちょっと怖い

それは普通です。初公開は誰でも少し身構えます。

でも今日は細部まで作り込んだ完成版を出す日ではありません。**Day 04** まで来た節目を外に出す日だと思ってよいです。

小さく共有してもよいし、公開範囲を絞ってもよいです。1回 URL を外に出す経験が次の伸びにつながります。

ここで確認したいこと

詰まったときに GitHub 側の状態・ローカル build・Vercel 連携権限・Project 設定・公開後の確認のどこで止まっているかを、この順番で見直せば十分です。

Step 12: 今日やったことを、自分の言葉で説明できるようにする(読む目安: 5分)

操作としてはここまでで十分です。でも理解としてもう一歩強くしたいなら次の4つを自分の言葉で説明できるとかなりよいです。

1. GitHub と Vercel の役割は違う

GitHub はコード履歴の保存先です。Vercel はそのコードを公開 URL に変える場所です。

Day 03 と Day 04 が分かれているのはこの役割が違うからです。

2. 公開の出発点は自分のリポジトリです

今日は見本コードを持っていったのではなくて Day 01 から積み上げた自分の `task-app` を公開しました。

ここが本当に大事です。

3. `npm run build` は公開前の確認になる

`build` を先に通すとエラー原因の切り分けがかなり楽になります。公開直前の一手として今後もずっと使えます。

4. 公開したら別環境でも見る

自分のいつものブラウザだけで終わらせず、シークレットウィンドウやスマホでも見ます。

これで初めて「本当に公開されている」と自信を持ちやすくなります。

覚えておきたいエッセンス

Day 04 の本質は単に Vercel のボタンを押すことではありません。

自分のアプリに、他人も開ける URL を持たせたことです。

覚えておきたいのはこの5つです。

- Day 04 は Day 03 の GitHub 保存を、そのまま公開へ橋渡しする日
- 公開するのは教材の見本ではなく、Day 01 から積み上げた自分の `task-app`
- Vercel へ行く前に `npm run build` を通すと公開フローがかなり安定します。
- 公開 URL はブラウザ1回確認で終わらせず、シークレットウィンドウやスマホでも見ます。
- 最初の公開 URL は遠慮せず SNS や身近な人に共有してよいです。

今日のチェックリスト

最後にこの Day の完了条件を自分で確認しておきましょう。

- ☐ GitHub に `task-app` が push 済みであることを確認しました。
- ☐ `npm run build` が通りました。
- ☐ Vercel に GitHub 連携でログインできました。
- ☐ `task-app` リポジトリを Import できました。
- ☐ Project 設定を確認して Deploy できました。
- ☐ 発行された URL をブラウザで開けました。
- ☐ シークレットウィンドウかスマホでも URL を確認しました。
- ☐ URL を誰かに共有した、または共有文の下書きを作りました。

全部埋まったら Day 04 は完了です。

Phase 1(環境構築・即公開)修了 ふりかえり

今日で Phase 1(環境構築・即公開)は終了です。

この4日でやったことを並べるとかなりちゃんと積み上がっています。

Day 01 でやったこと

- `task-app` の土台を立ち上げました。
- Next.js アプリを最初の画面まで動かしました。
- design token を入れて見た目の芯を整え始めました。

Day 02 でやったこと

- ダッシュボードに自分だけのメッセージを入れました。
- ただの教材見本から自分のプロダクトっぽさを出し始めました。
- Server Component を基準にする考え方にも触れました。

Day 03 でやったこと

- GitHub にコードを保存しました。
- 履歴として積み上げられる状態を作りました。
- 次の公開に向かうための保存先を整えました。

Day 04 でやったこと

- Vercel でネット公開しました。
- 公開 URL を持ちました。
- スマホや他人の環境からも開ける状態にしました。

つまり Phase 1 で手に入ったもの

Phase 1 の4日間で、次に進むための土台がそろいました。

- 手元で動きます。
- 自分の名前やメッセージが表示されます。
- GitHub に保存されています。
- URL で外から見られます。

これってただの「勉強した」より一段強いです。

自分のプロダクトの最初の輪郭を、外に出せるところまで持ってきたということです。

ここまで来たら Day 05 からの Phase 2(ログイン機能)も、「教材を消化する」ではなくて「この公開済みアプリをもっと育てる」感覚で進めやすくなります。

つまずきポイント

`npm run build` が `Type error:` で止まる

原因

型の書き間違いが残っています。`npm run dev` は型検査をしません。

解決方法

赤い行のうちいちばん上だけを読みます。`Type error:` の1つ上の行にファイル名と行番号、`Type error:` の後ろに理由、その下にコードの抜粋が出ます。そこを直して `npm run build` をやり直してください。

ページ数が教材の `(6/6)` と違う

原因

作ったページの数、その時点の状態で変わります。

解決方法

数字は合っていなくても問題ありません。`Collecting build traces` の表示だけでは判定せず、エラーなく最後まで完走してターミナルが次の入力待ちへ戻れば成功です。

Vercel の Import 画面に `task-app` が出てこない

原因

Vercel の GitHub App に、そのリポジトリを見る権限を渡していません。

解決方法

`Adjust GitHub App Permissions` から `task-app` を選んで許可します。

Private リポジトリの場合は特に起きやすい問題です。

Vercel の Deploy がビルドエラーで止まる

原因

手元で通っていないコードを送っているか、環境変数が足りません。

解決方法

まず Step 2 の `npm run build` が手元で通るか確かめます。通る場合は Environment Variables に `DATABASE_URL` と `JWT_SECRET` が入っているかを確認してください。

公開 URL の画面が手元で直したはずの内容になっていない

原因

GitHub へ push していません。Vercel はコードを手元からではなく、GitHub から取りに行きます。

解決方法

`git status` で未コミットの差分を確かめ、`git add` → `git commit` → `git push` まで進めます。連携先ブランチへの push でデプロイが自動開始するため、Deployments で最新コミットの結果を確認してください。

公開先だけビルドの結果が手元と違う

原因

Node のバージョンが手元と公開先でそろっていません。

解決方法

Day 03 の Step 7 で `package.json` を add・push したか確認します。公開先の Node のバージョンは `package.json` の `engines.node` で決まります。それでも違う場合は、Vercel の Settings から Build and Deployment の Node.js Version を確認してください。

公開 URL をスマホで開くと表示が崩れる

原因

画面幅が狭いときの指定が入っていません。

解決方法

今日は直さなくても問題ありません。崩れた場所をメモしておき、以降の Day でレイアウトを作るときに直します。

`DATABASE_URL` を GitHub に置いてしまった

原因

本物の値と見本の区別が付いていません。

解決方法

本物は Vercel の Environment Variables にだけ置きます。GitHub には `.env.example` の見本だけを送ります。すでに送っていた場合は、Day 03 の Step 10 の手順で鍵を作り直してください。

今日学んだ用語

用語	意味
Vercel	Next.js アプリを GitHub から取ってきて公開してくれるホスティングサービス
デプロイ	アプリを公開用のサーバーに置いて誰でも開ける状態にすること
ビルド	ソースコードを本番用の形に変換する処理。 <code>npm run build</code> で走る
Production Deploy	本番用として公開する Vercel の操作。ここで発行された URL が人へ送る URL になる
環境変数	接続先やパスワードを、コードの外から渡す仕組み
接続文字列	データベースの場所とログイン情報を1本にまとめた文字列
Neon	ブラウザだけで PostgreSQL を1つ用意できるサービス。今日は公開先から使う DB として使う
シークレットウィンドウ	ログイン状態やキャッシュを持たない状態でページを開くブラウザの機能

応用課題: ローカルの変更と公開中の画面を見分ける

公開URLと手元の画面を並べ、どちらのコードを見ているか確かめましょう。公開が完了してから取り組む課題です。ローカル側は `npm run dev` で起動します。

`npm start` が動いている場合は、そのターミナルで `Ctrl+C` を押してから起動し直してください。`npm start` はビルド済みの画面を表示するため、ソースの編集だけでは表示が変わりません。

(1) 公開URLとローカルの `/dashboard` を別々のタブで開きます。両方に表示されている自分の名前をメモします。

(2) `src/app/dashboard/page.tsx` の `dashboardOwner.name` を一時的に別の名前へ変えて保存します。この変更のコミット・pushはしません。

(3) ローカルと公開URLの両方を再読み込みします。ローカルだけが新しい名前になり、公開URLは元の名前なら成功です。差がない場合は各タブのアドレスを確かめます。

(4) 名前をメモした値へ戻して保存します。ローカルも元の表示へ戻ったことを確認します。公開先へ変更を送る操作は不要です。

理解チェック

今日の作業を思い出しながら答えてみてください。答えは各問のすぐ下にあります。

Q1. `npm run build` は `npm run dev` と何が違いますか。

Day 04: ネットに公開

A. `dev` は画面を動かして確かめるためのコマンドで、型検査は行いません。

`build` は本番用のファイルを作る途中で、プロジェクト全体の型を確認します。

Vercel も公開時に `build` を実行するため先に手元で通しておくのと型の誤りを公開前に直せます。

Q2. 手元のコードを直したあと、GitHub へ push せずに Vercel で再デプロイすると公開 URL はどうなりますか。

A. 直す前の内容のままです。Vercel がコードを取りに行く先は自分のパソコンではなく GitHub なので push していない変更は公開されたアプリに出てきません。「公開先だけ古い」と感じたときはまず `git status` で送り忘れを疑ってください。

Q3. `DATABASE_URL` と `JWT_SECRET` を、GitHub ではなく Vercel の Environment Variables に登録するのはなぜですか。

A. 本物の値をリポジトリに入れず、見本だけを共有するという Day 03 の線引きを、公開先でもそのまま守るためです。GitHub に置くとリポジトリを見られる相手へ接続先ごと渡すことになります。Vercel の Environment Variables は公開されないなので本物はそちらだけに置きます。

次回予告

Phase 1 の土台はここで完成です。次からは Phase 2 (ログイン機能)に入ります。

Day 05 ではログイン UI の導入から始めます。

公開 URL を持った今の `task-app` に、「誰が使うアプリなのか」という入口を足していく段階です。

ここから先は見た目だけではなく、実際に使うための機能を作っていきます。

次はログイン画面をつかってこのアプリにちゃんとした入口を用意します。

次に読むもの

- 前の日: Day 03^{*5}
- 次の日: Day 05^{*6}
- 全体の地図: 学びのロードマップ^{*7}
- 目次: カリキュラム目次^{*8}
- 詰まったとき: トラブルシューティング^{*9}
- 言葉の意味: 用語集^{*10}

奥付

タイトル	Day 04: ネットに公開
著者	磯貝光佑
版	第1版(2026年9月19日)

© 2026 磯貝光佑

-
5. https://drive.google.com/file/d/1v0eZADJHtUknINZOudHy4oBMpfk4t_R0/view?usp=drivesdk
 6. <https://drive.google.com/file/d/18Mq1zmBcLFNfxATqwHoeFrtzeGldT4UZ/view?usp=drivesdk>
 7. https://drive.google.com/file/d/10py_p-6ry4RDlxBSyzPn4rtzZe74ND-T/view?usp=drivesdk
 8. <https://drive.google.com/file/d/19g0cKLajEWi7s9LeWF804DB9j5kK1dt4/view?usp=drivesdk>
 9. <https://drive.google.com/file/d/1OyjFWiGvZBm9QKcnKiougPZidjsn7kqR/view?usp=drivesdk>
 10. <https://drive.google.com/file/d/1RtElBpzkvXsoTpj4CTKPko8jnQDcPsh/view?usp=drivesdk>

Day 04: ネットに公開

本書は購入者個人の利用に限ります。本書に掲載されたコードの写経、および自分のプロジェクトへの転用・改変は自由です。本書(PDF・Markdown)および付属コードの再配布・転売・第三者との共有は、形態を問わず禁止します。